Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 4

«API и BLoC (часть 1)»

По дисциплине “Кроссплатформенные средства разработки”

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель: освоить работу с API и HTTP-запросами во Flutter, а также научиться управлять состоянием приложения с помощью архитектуры BLoC.

https://github.com/kukirmash/kukirmash_weather_app

Ход работы:

1) Установка зависимостей

Для выполнения работы добавлены Dio для HTTP-запросов, flutter_bloc и equatable для управления состоянием, talker_dio_logger и talker_bloc_logger для логирования, а также json_annotation, json_serializable и build_runner для генерации кода сериализации.

pubspec.yaml

```
dependencies:
  dio: ^5.11.1
  equatable: ^3.0.0
  flutter_bloc: ^9.1.1
  json_annotation: ^4.12.0
  talker_dio_logger: ^5.1.20
  talker_bloc_logger: ^5.1.20

dev_dependencies:
  build_runner: ^2.16.1
  json_serializable: ^6.14.1
```

2) Выбор API и настройка HTTP-клиента

В качестве внешнего API выбран Open-Meteo — открытый сервис прогноза погоды, не требующий API-ключа. Точка расчёта — Чебоксары (широта 56.13, долгота 47.25). Dio настраивается в файле data/dio/set_up.dart: задаются baseUrl, координаты, часовой пояс, таймауты и перехватчик логирования.

```
GET https://api.open-meteo.com/v1/forecast
  ?latitude=56.13
  &longitude=47.25
  &timezone=auto
  &daily=weather_code,temperature_2m_max,temperature_2m_min,
         apparent_temperature_max,apparent_temperature_min,
         relative_humidity_2m_mean,surface_pressure_mean,
         wind_speed_10m_max,precipitation_probability_max,
         sunrise,sunset,uv_index_max
  &forecast_days=7
```

lib/data/dio/set_up.dart

```
void setUpDio() {
  dio.options
    ..baseUrl = 'https://api.open-meteo.com/v1'
    ..connectTimeout = const Duration(seconds: 10)
    ..receiveTimeout = const Duration(seconds: 15)
```

```
    ..queryParameters.addAll({
      'latitude': kLatitude,
      'longitude': kLongitude,
      'timezone': 'auto',
    });

  dio.interceptors.add(
    TalkerDioLogger(
      talker: talker,
      settings: const TalkerDioLoggerSettings(
        printRequestData: true,
        printRequestHeaders: true,
      ),
    ),
  );
}
```

3) Модели данных

Ответ Open-Meteo содержит не массив объектов, а набор параллельных массивов: time, temperature_2m_max, weather_code и другие. DTO ForecastResponse разбирает этот ответ, а метод toDailyForecasts() склеивает колонки в список моделей DailyForecast. Для генерации методов fromJson и toJson используется json_serializable.

lib/data/forecast/forecast_response.dart (фрагмент)

```
@JsonSerializable()
class ForecastResponse {
  const ForecastResponse({this.daily, this.hourly});

  final DailyBlock? daily;
  final HourlyBlock? hourly;

  factory ForecastResponse.fromJson(Map<String, dynamic> json) =>
      _$ForecastResponseFromJson(json);

  List<DailyForecast> toDailyForecasts() {
    final block = daily;
    if (block == null) return const [];

    return List.generate(block.time.length, (index) {
      return DailyForecast(
        id: block.time[index],
        date: DateTime.parse(block.time[index]),
        weatherCode: block.weatherCode[index].toInt(),
        temperatureMax: block.temperatureMax[index].toDouble(),
        temperatureMin: block.temperatureMin[index].toDouble(),
        relativeHumidity: block.relativeHumidity[index].round(),
        // остальные показатели...
      );
    });
  }
}
```

Поля с именами, отличающимися от Dart-стиля, помечены аннотацией @JsonKey. Например, поле relative_humidity_2m_mean объявлено следующим образом:

```
@JsonKey(name: 'relative_humidity_2m_mean')
final List<num> relativeHumidity;

@JsonKey(name: 'temperature_2m_max')
final List<num> temperatureMax;
```

4) Репозиторий прогноза

Репозиторий скрывает детали HTTP-запроса от блока. Интерфейс содержит метод получения прогноза на неделю, а реализация формирует параметры запроса, разбирает ответ и преобразует ошибки Dio в понятное пользователю исключение ForecastException.

lib/data/forecast/forecast_repository.dart (фрагмент)

```
abstract interface class ForecastRepositoryInterface {
  Future<List<DailyForecast>> getWeekForecast();
  Future<DayDetails> getDayDetails(String id);
}

class ForecastRepository implements ForecastRepositoryInterface {
  ForecastRepository({required this.dio});

  final Dio dio;

  @override
  Future<List<DailyForecast>> getWeekForecast() async {
    try {
      final response = await dio.get<Map<String, dynamic>>(
        Endpoints.forecast,
        queryParameters: {
          'daily': _dailyParameters,
          'forecast_days': 7,
        },
      );
      final data = ForecastResponse.fromJson(response.data!);
      final days = data.toDailyForecasts();
      if (days.isEmpty) {
        throw const ForecastException('Сервер не прислал прогноз.');
      }
      return days;
    } on DioException catch (exception) {
      throw ForecastException(_dioMessage(exception));
    }
  }
}
```

5) BLoC первого экрана

Состояние экрана описано sealed-классами: HomeInitial, HomeLoading, HomeLoaded и HomeError. Событие HomeLoadRequested запускает загрузку. Если данные уже были загружены, при обновлении экран не переключается на индикатор загрузки, а продолжает показывать старые данные.

lib/app/features/home/bloc/home_state.dart

```
sealed class HomeState extends Equatable {
  const HomeState();
}

final class HomeInitial extends HomeState { const HomeInitial(); }
final class HomeLoading extends HomeState { const HomeLoading(); }

final class HomeLoaded extends HomeState {
  const HomeLoaded({
    required this.days,
    required this.updatedAt,
    this.refreshError,
  });
```

```
  final List<DailyForecast> days;
  final DateTime updatedAt;
  final Object? refreshError;
}

final class HomeError extends HomeState {
  const HomeError({required this.exception});
  final Object exception;
}
```

lib/app/features/home/bloc/home_bloc.dart (фрагмент)

```
class HomeBloc extends Bloc<HomeEvent, HomeState> {
  HomeBloc(this._forecastRepository) : super(HomeInitial()) {
    on<HomeLoadRequested>(_onLoadRequested);
  }

  final ForecastRepositoryInterface _forecastRepository;

  Future<void> _onLoadRequested(
    HomeLoadRequested event,
    Emitter<HomeState> emit,
  ) async {
    if (state is! HomeLoaded) emit(const HomeLoading());

    try {
      final days = await _forecastRepository.getWeekForecast();
      if (isClosed) return;
      emit(HomeLoaded(days: days, updatedAt: DateTime.now()));
    } catch (exception, stackTrace) {
      talker.handle(exception, stackTrace);
      if (isClosed) return;
      emit(HomeError(exception: exception));
    } finally {
      event.completer?.complete();
    }
  }
}
```

6) Регистрация зависимостей и интеграция с экраном

Репозиторий и блок регистрируются в getIt. Экран создаёт блок через BlocProvider, сразу отправляет событие загрузки и отображает состояние с помощью BlocBuilder. Pull-to-refresh передаёт в событие Completer, поэтому индикатор обновления завершается только после ответа сервера.

lib/di/di.dart (фрагмент)

```
getIt.registerLazySingleton<ForecastRepositoryInterface>(
  () => ForecastRepository(dio: dio),
);
getIt.registerFactory<HomeBloc>(
  () => HomeBloc(getIt<ForecastRepositoryInterface>()),
);
```

lib/app/features/home/home_screen.dart (фрагмент)

```
BlocBuilder<HomeBloc, HomeState>(
  builder: (context, state) {
    return switch (state) {
      HomeInitial() || HomeLoading() => const AppLoading(),
      HomeError(:final exception) => AppError(
        message: exception.toString(),
        onRetry: () => context.read<HomeBloc>().add(
          const HomeLoadRequested(),
        ),
      ),
```

```
      HomeLoaded(:final days, :final updatedAt, :final refreshError) =>
        _HomeContent(
          days: days,
          updatedAt: updatedAt,
          refreshError: refreshError,
        ),
    };
  },
)
```

7) Результат

Первый экран получает актуальный прогноз из Open-Meteo, показывает состояние загрузки и ошибки, поддерживает повторную загрузку и pull-to-refresh. На рисунке представлен экран с данными, полученными от API.

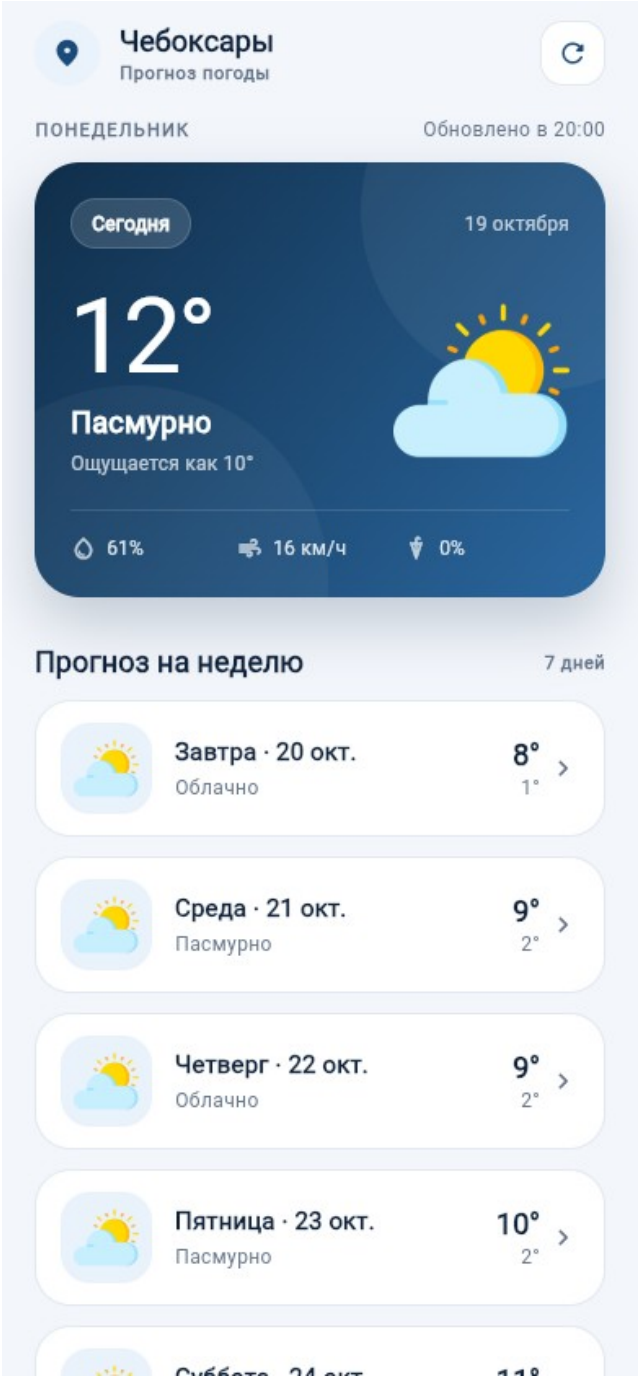


Рисунок 1 — первый экран с данными Open-Meteo

Дополнительно выполнены проверки flutter analyze и flutter test. Анализатор не выявил ошибок, все 13 тестов проходят.

Вывод:

в ходе работы был настроен HTTP-клиент Dio, реализованы модели и репозиторий для Open-Meteo, создан HomeBloc и выполнена интеграция загрузки данных с первым экраном.